

Encryption and Key Management (Group Task)

4/7/2025

by: Sara kedir - Taef Alzahrani

Objective:

To gain hands-on experience in data protection by applying symmetric encryption (AES) and asymmetric encryption (RSA), implementing encryption, decryption, and key rotation processes on realistic dummy data. Requirements for the report:

- Develop a script to generate an RSA key pair (public and private keys).
- The key size must be at least 2048 bits.
- Store the keys in the keys/ directory in .pem format.
- Use the AES algorithm in CBC (Cipher Block Chaining) mode.
- Generate a random AES key and IV.
- Encrypt the following files:
 - The contents of fake_data.json
 - The contents of fake_message.txt

Encrypt the AES key itself using the RSA public key. Store the outputs (encrypted content, encrypted AES key, and IV) in the encrypted/ directory.

- Use the RSA private key to decrypt the AES key.
- Use the decrypted AES key and IV to decrypt the data.
- Ensure that the decrypted data matches the original data exactly.
- Store the private key encrypted with a password using BestAvailableEncryption.
- Implement a simple key rotation mechanism that preserves old keys while generating new ones.
- Implement access control for the keys (e.g., the private key is only loaded after providing the correct password).

Required Testing:

Each trainee must use the following files for testing:

- test_data/fake_data.json: Contains dummy card and customer data.
- test_data/fake_message.txt: Contains a dummy confidential administrative message.

Execute the full encryption and decryption workflow on this data. Verify that the output after decryption matches the original file content. Attach a report, screenshots, or logs demonstrating the successful operation.

Introduction

Encryption is the process of converting readable data (plaintext) into an unintelligible form (ciphertext) to prevent unauthorized access. It can later be reversed through decryption using a cryptographic key. Modern standards like AES and RSA, recommended by agencies such as NIST, ensure that sensitive data, whether it is stored on a device or transmitted over networks, remains confidential and protected from tampering

Programming Details:

Programming Language Used: Python

Library Used: cryptography

Step 1: Develop a script to generate an RSA key pair (public and private keys).

1. What is RSA?

RSA (Rivest–Shamir–Adleman) is one of the most widely used asymmetric encryption algorithms. It relies on a key pair: a public key for encryption and a private key for decryption. RSA is used to securely exchange data over untrusted networks and is the foundation of many security protocols like SSL/TLS.

2. What is a Public Key and a Private Key?

Public Key:

This key can be shared with anyone. It's used to encrypt data. Anyone who has your public key can send you encrypted messages, but only you can decrypt them using your private key.

Private Key:

This key must be kept secret. It is used to decrypt data that was encrypted with the matching public key. In our task, we also apply password protection to the private key for added security.

3. How this relates to the task:

In the encryption project, RSA is used to encrypt the AES symmetric key. Since RSA is not efficient for encrypting large data, we use it only to protect the AES key, and AES handles the actual file encryption. This combination is called hybrid encryption, a secure and efficient way to handle data protection.

4. How we implemented it:

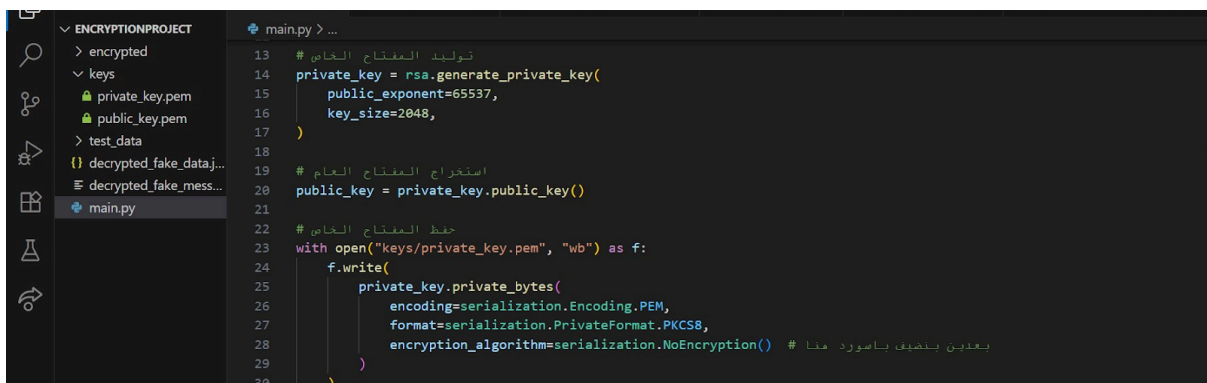
We used Python and the **cryptography** library to generate an RSA key pair with a key size of 2048 bits.

Here's what we did in the script:

Here's what we did in the script:

- Used `rsa.generate_private_key()` to create the private key.
- Extracted the public key using `private_key.public_key()`.
- Saved both keys in `.pem` format inside the `keys/` directory.
- At this step, the private key is saved without password encryption, which we handle later in the task.

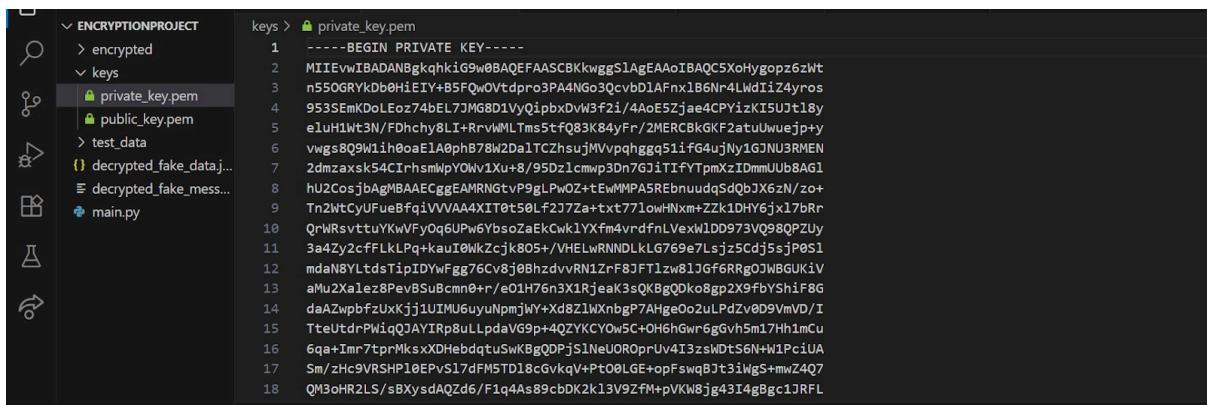
(screenshots clarify the steps)



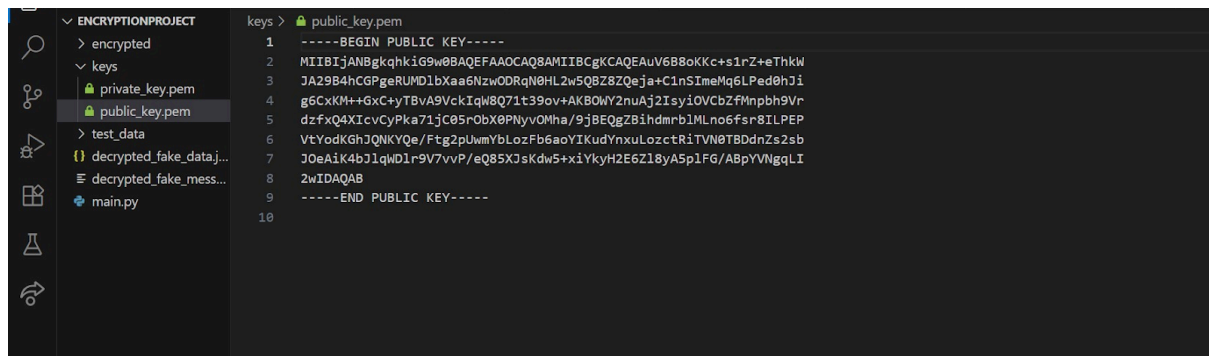
```
13 # توليد المفتاح الخاص
14 private_key = rsa.generate_private_key(
15     public_exponent=65537,
16     key_size=2048,
17 )
18
19 # استخراج المفتاح العام
20 public_key = private_key.public_key()
21
22 # حفظ المفتاح الخاص
23 with open("keys/private_key.pem", "wb") as f:
24     f.write(
25         private_key.private_bytes(
26             encoding=serialization.Encoding.PEM,
27             format=serialization.PrivateFormat.PKCS8,
28             encryption_algorithm=serialization.NoEncryption() # نعلن بتشفير باسورد هنا
29         )
30     )
```



```
26
27     encoding=serialization.Encoding.PEM,
28     format=serialization.PrivateFormat.PKCS8,
29     encryption_algorithm=serialization.NoEncryption() # نعلن بتشفير باسورد هنا
30 )
31
32 # حفظ المفتاح العام
33 with open("keys/public_key.pem", "wb") as f:
34     f.write(
35         public_key.public_bytes(
36             encoding=serialization.Encoding.PEM,
37             format=serialization.PublicFormat.SubjectPublicKeyInfo
38         )
39     )
40
41 print("تم توليد وحفظ المفاتيح بنجاح")
42
```



```
keys > private_key.pem
1 -----BEGIN PRIVATE KEY-----
2 MIIEvwIBADANBgkqhkiG9w0BAQEFAASCBAkkgwS1AgEAAoIBAQC5XoHygopz6Zwt
3 n550GRYkDb0HiEiY+B5FQwOVtdpro3PA4Ngo3QcVbD1AFnx1B6Nr4LWdIiZ4yros
4 953SEmKDoLEoz74bEL7JMG8D1VyQipbxDvW3f2i/4AoE5Zjae4CPYizKI5Ujtl8y
5 eluH1Wt3N/FDhchy8LI+RrvvMLTms5tfQ83K84yFr/2MERCbKGKf2atuUwuejp+y
6 vWgs8Q9W1ih0aE1A0phB78W2Da1TCZhsuJMVvpqhggq51ifG4ujNy1G3NU3RMEN
7 2dmzaxsk54CIRhsmWpYOWv1Xu+8/95Dz1cmwp3Dn7G3iITfYTpMxZIDmmUUb8AG1
8 hU2CosjbAgMBAAECggEAMRNGtVp9gLpWQZ+EtWMPA5REbnuudqSdQbJX6zN/zo+
9 Tn2WtCyUFueBfqiVVVAA4XIT0t50Lf2J7Za+txt77lowHNxm+ZZK1DHY6jx17bRr
10 QrWRsvttuYKwVfyQ6UPw6YbsoZaEkCwk1YXfm4vrdfnLvexw1DD973VQ98QPZuy
11 3a4Zy2cFFLkLPg+kauI0WkZcjk805+/VHELwRNNDLKL769e7Lsjz5Cdj5sJP0S1
12 mdaN8YLtdsTipIDYwFgg76Cv8j0BhzdvVRN1ZrF8JFT1zw81JGf6RRgOJWBGUKiV
13 aMu2Xalez8PevBSuBcmn0+r/eO1H76n3X1RjeaK3sQKBgQDko8gp2X9fbYShiF8G
14 daAZwpbfzUxKjj1UIMU6uyuNpmjWY+Xd8Z1WxnbGp77AHgeOo2uLPdZv0D9VmVD/I
15 TteUtdrPWiqQJAYIRp8uLLpdaVG9p+4QZYKYOW5C+OH6hGwr6gGvh5m17Hh1mCu
16 6qa+Imr7tprMksxXDHebdqtuSwKBQDPjS1NeUOROpUv4I3zsdWtS6N+WiPciUA
17 Sm/zHc9VRSHp10EPvS17dFM5TD18cGvKqV+Pt00LGE+opFswqB3t3iWgS+mwZ4Q7
18 QM3oHR2LS/sBXysdAQZd6/F1q4As89cbDK2k13V9ZfH+pVKW8Jg43I4g8Gc1JRFL
```



Step 2: Use the AES algorithm in CBC mode & generate AES key and IV

1. What is AES?

AES (Advanced Encryption Standard) is a symmetric encryption algorithm, meaning it uses the same key for both encryption and decryption. It is widely used for securing sensitive data due to its strength and efficiency. AES operates on fixed-size blocks of data (usually 128 bits) and supports key sizes of 128, 192, or 256 bits. In our project, we used a 256-bit AES key for stronger encryption.

2. What is CBC (Cipher Block Chaining) mode?

CBC is a mode of operation for block ciphers like AES. Instead of encrypting each block of plaintext independently (which could lead to patterns), CBC improves security by chaining the encryption:

Each plaintext block is XOR'ed with the previous ciphertext block before encryption.

For the first block, there's no previous ciphertext, so we use a random Initialization Vector (IV).

This method ensures that even if two plaintext blocks are identical, the resulting ciphertexts will be different.

3. What is an IV (Initialization Vector)?

The IV is a random, non-secret value used to start the CBC encryption chain. It must:

- Be unique for every encryption operation.
- Be exactly 16 bytes (same as the AES block size).
- Be saved along with the ciphertext, since it's needed for decryption.

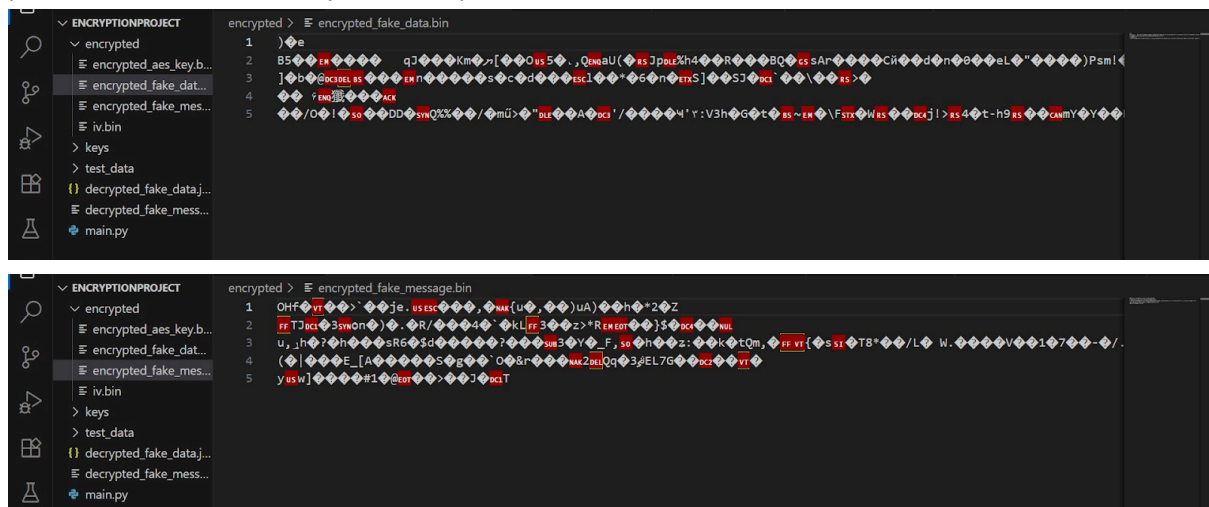
4. What we did in the project:

- We generated a random 256-bit AES key using Python's `secrets.token_bytes(32)`.
- We generated a 16-byte IV using `secrets.token_bytes(16)`.
- We used AES with CBC mode from the cryptography library to encrypt:
- `fake_data.json` (contains dummy customer data).
- `fake_message.txt` (contains a dummy admin message).
- We padded the data using PKCS7 to ensure its length aligns with the block size required for AES encryption.
- We saved the encrypted files and the IV inside the `encrypted/` directory.

Why this step is important :

This step ensures that the actual data (files) is encrypted efficiently and securely. Since AES is fast and suitable for large data, it's used for encrypting file contents directly. The IV ensures randomness and protects against repeated patterns in encrypted files.

(Screenshots show encrypted files)



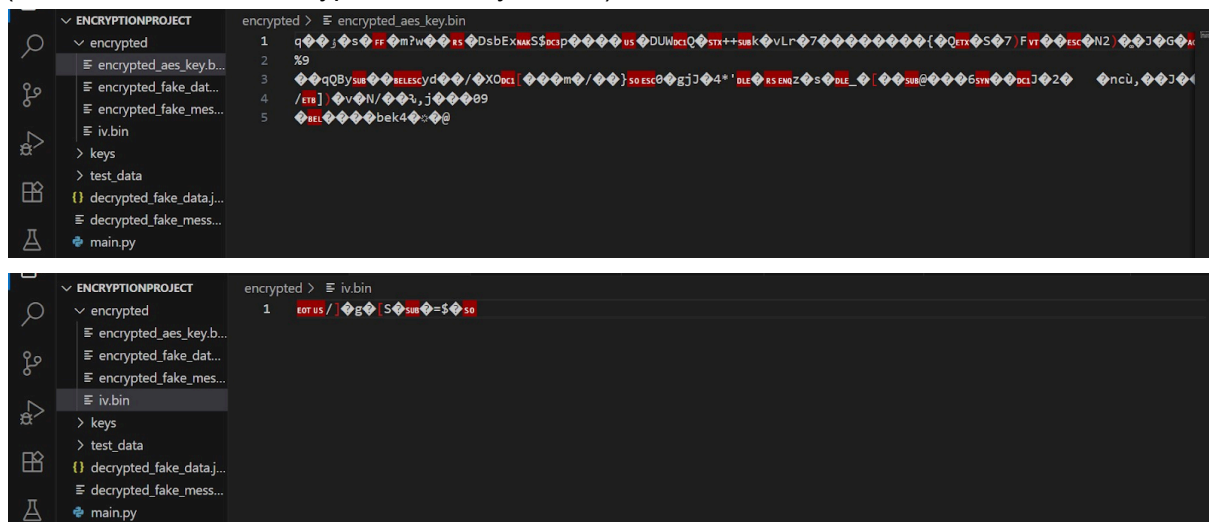
Step 3: Encrypting and Storing the AES Key and IV

After encrypting the files using AES, we needed a way to secure the AES key itself. To do that, we:

- Encrypted the AES key using the RSA public key, so that no one can access the AES key without having the RSA private key.
- Stored the following securely in the encrypted/ directory:
 - The AES-encrypted content (encrypted_fake_data.bin, encrypted_fake_message.bin)
 - The RSA-encrypted AES key (encrypted_aes_key.bin)
 - The IV (Initialization Vector) used in AES (iv.bin)

Storing the AES key encrypted with RSA provides an extra layer of protection, ensuring that only authorized users with the RSA private key can decrypt and use it.

(Screenshots show encrypted AES key and IV)



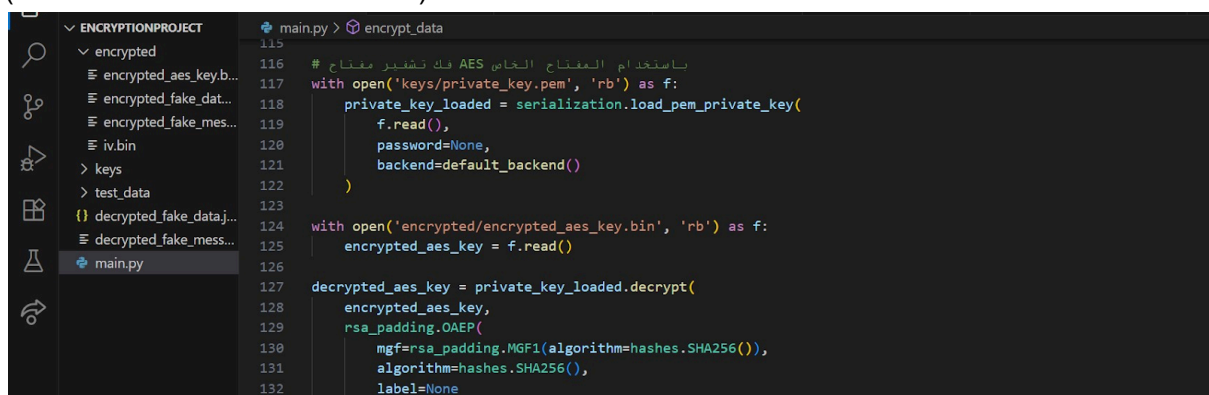
Step 4: Decrypting the AES Key Using RSA Private Key

To reverse the process and recover the AES key:

We used the RSA private key to decrypt the RSA-encrypted AES key.

This step ensures that only someone who holds the private key (and enters the correct password if it's protected) can decrypt the AES key and access the data.

(A screenshot of the code used)



Sept 5: Decrypting the Files with AES

After recovering the AES key and IV, we:

Decrypted the contents of both encrypted_fake_data.bin and encrypted_fake_message.bin.

Used the same AES key and IV to make sure the decryption matched the original encryption configuration.

Verifying the Integrity of the Data

We compared the decrypted data with the original files (fake_data.json and fake_message.txt) to ensure they matched exactly. This proves:

The AES and RSA encryption/decryption processes worked correctly.

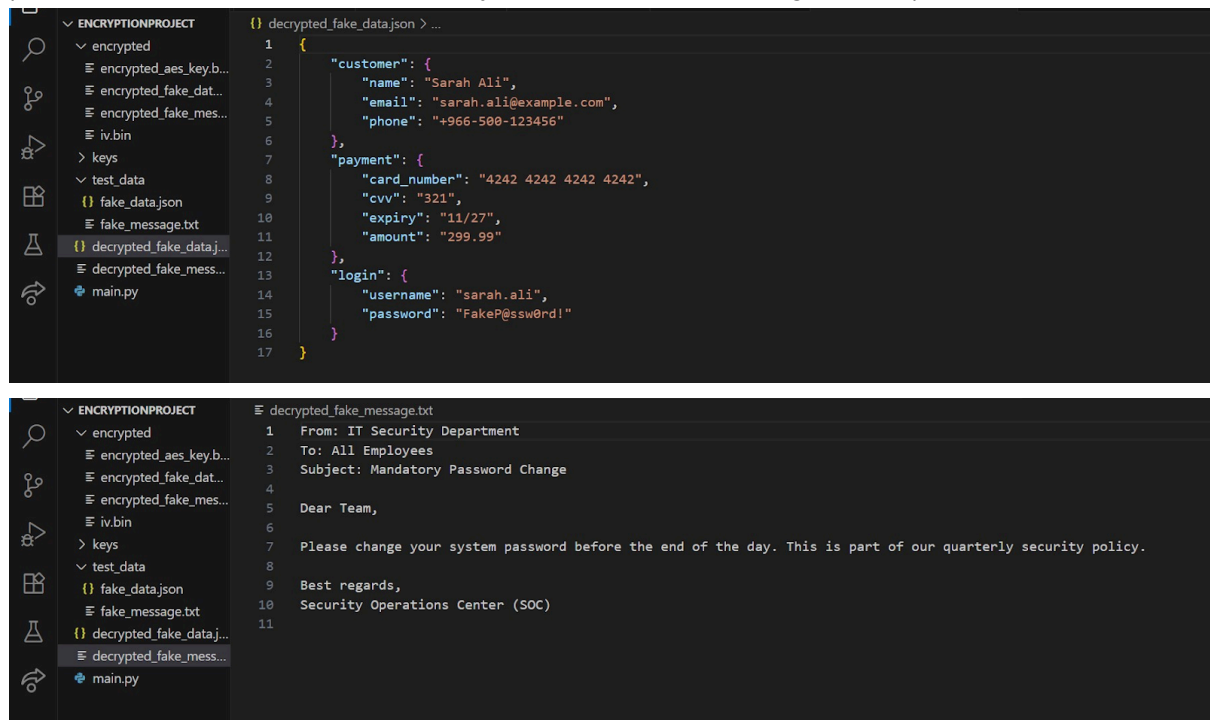
No data was lost or corrupted.

The padding (PKCS7) and mode (CBC) were applied and reversed properly.

Why This Step is Important:

Encrypting the AES key with RSA ensures secure key distribution, and verifying decrypted data guarantees that data integrity and confidentiality are preserved.

(The screenshots show that the decrypted files match the original files)



```
{} decrypted_fake_data.json > ...
1  {
2      "customer": {
3          "name": "Sarah Ali",
4          "email": "sarah.ali@example.com",
5          "phone": "+966-500-123456"
6      },
7      "payment": {
8          "card_number": "4242 4242 4242 4242",
9          "cvv": "321",
10         "expiry": "11/27",
11         "amount": "299.99"
12     },
13     "login": {
14         "username": "sarah.ali",
15         "password": "FakeP@ssw0rd!"
16     }
17 }
```

```
{} decrypted_fake_message.txt
1  From: IT Security Department
2  To: All Employees
3  Subject: Mandatory Password Change
4
5  Dear Team,
6
7  Please change your system password before the end of the day. This is part of our quarterly security policy.
8
9  Best regards,
10 Security Operations Center (SOC)
11
```

Ensure that the decrypted data matches the original data exactly.

What does "Ensure that the decrypted data matches the original data exactly" mean in encryption?

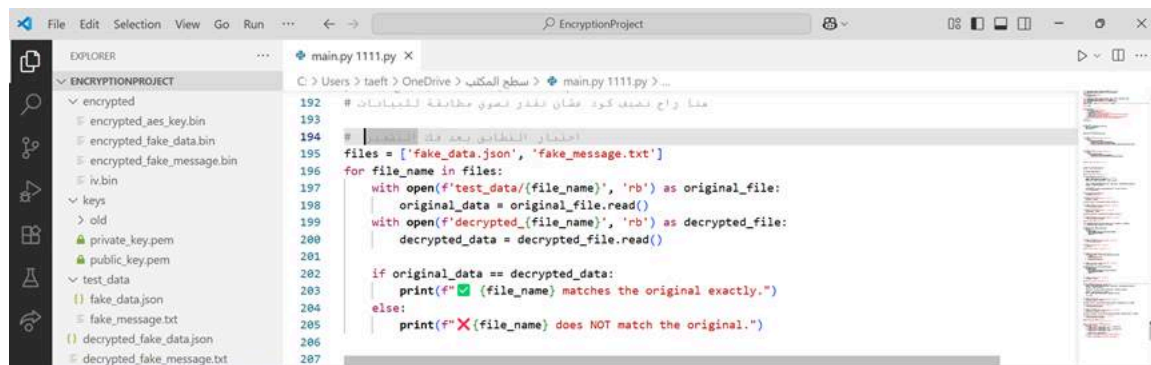
It means verifying that the data after decryption is exactly the same as the original data before encryption.

Its Purpose:

- Ensures data integrity and confirms that the encryption and decryption processes did not alter the data.
- Helps detect any corruption or tampering during storage or transmission.
- It is a critical step in security systems to maintain data trustworthiness after decryption.

```
files = ['fake_data.json', 'fake_message.txt']
for file_name in files:
    with open(f'test_data/{file_name}', 'rb') as original_file:
        original_data = original_file.read()
        with open(f'decrypted_{file_name}', 'rb') as decrypted_file:
            decrypted_data = decrypted_file.read()

    if original_data == decrypted_data:
        print(f"{file_name} matches the original exactly.")
    else:
        print(f"{file_name} does NOT match the original.")
```



Store the private key encrypted with a password using BestAvailableEncryption.

-It is a tool in Python's **cryptography** library used to encrypt your private key when saving it, using the strongest encryption algorithm available in the library.

Its Purpose:

- It protects your private key so that even if someone accesses the file, they cannot decrypt or use the key without the password.
- It is a strong security practice to protect private keys during storage, especially in sensitive projects or production environments.

-In this step, we used BestAvailableEncryption to set a password that restricts the use of the keys and allows them to be used only by the user

encryption_algorithm=serialization.BestAvailableEncryption(password_input)



Implement a simple key rotation mechanism that preserves old keys while generating new ones

What is Key Rotation?

It is a mechanism in encryption used to periodically replace old encryption keys with new ones while preserving old keys so that data encrypted with old keys can still be decrypted.

Its Purpose:

- Minimizes damage in case of key leakage by regularly replacing keys.
- Improves security by not relying on a single key for too long.
- Helps meet security policies and compliance requirements within organizations.

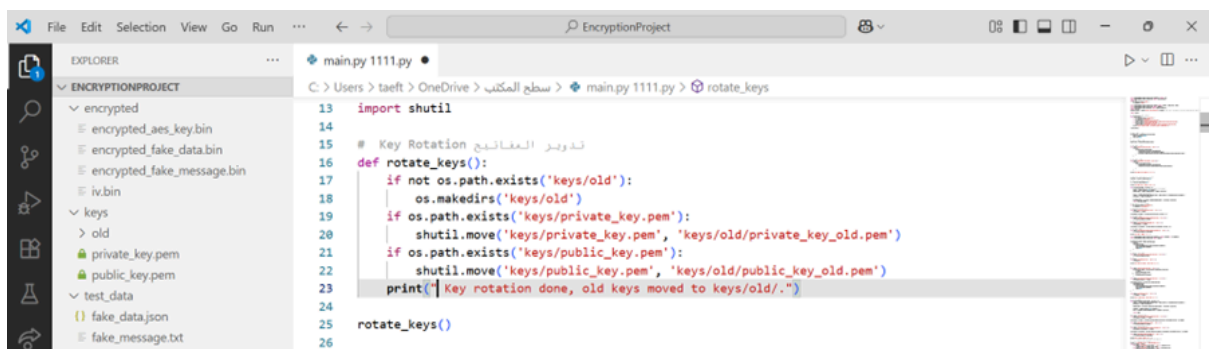
Simple example:

- The current key (Key v1) encrypts data.
- During rotation, a new key (Key v2) is generated to encrypt new data.
- Key v1 is preserved to decrypt old data encrypted with it.

import shutil

```
def rotate_keys():
    if not os.path.exists('keys/old'):
        os.makedirs('keys/old')
    if os.path.exists('keys/private_key.pem'):
        shutil.move('keys/private_key.pem', 'keys/old/private_key_old.pem')
    if os.path.exists('keys/public_key.pem'):
        shutil.move('keys/public_key.pem', 'keys/old/public_key_old.pem')
    print("Key rotation done, old keys moved to keys/old/.")
```

rotate_keys()



What is Access Control in Encryption?

It is a mechanism to control access to encrypted keys (e.g., the private key) so that the key can only be used after providing the correct password.

Its Purpose:

- Protects the private key from unauthorized access, even if someone gains access to the file.
- Enhances secure key management practices by ensuring keys are not loaded into memory or used without user authentication (e.g., via a password).
- It is a fundamental part of secure key storage and handling in encryption systems.

Here, the password is requested from the user manually to protect it and to avoid hardcoding it in the script.



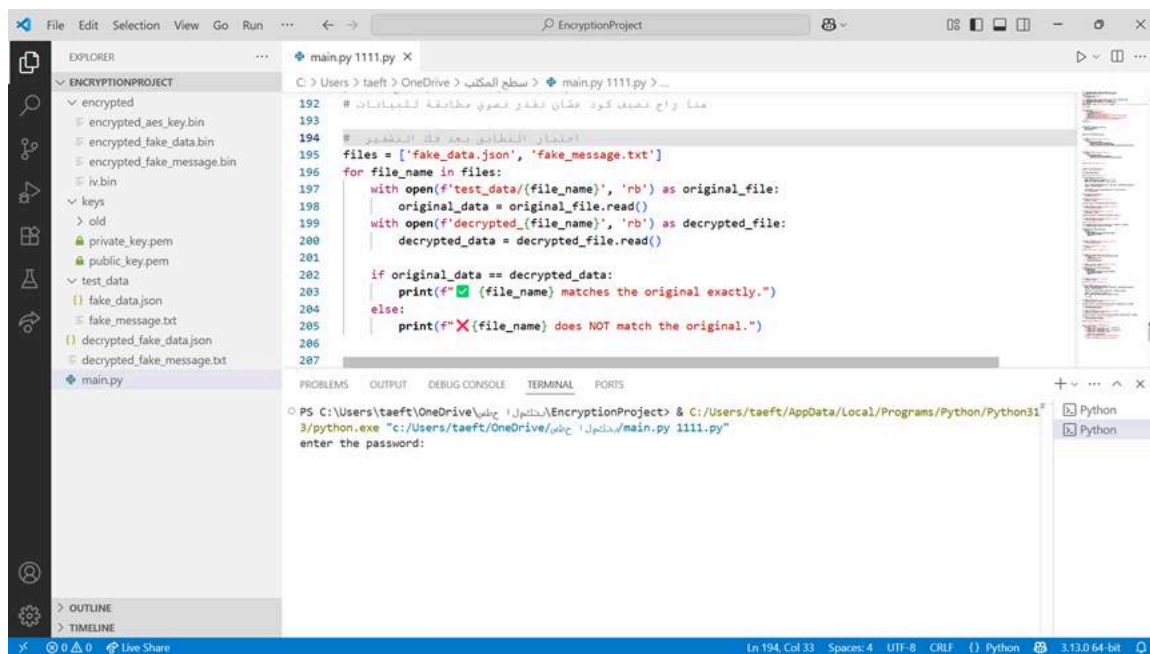
Here, the private key is loaded using the password:

- If the password is correct, the key will be decrypted and loaded.
- If the password is incorrect, an error will appear, and the key will not be loaded.



Now we will see the outputs of this code and how it is executed.

when running the code, a password is requested.



The screenshot shows a Visual Studio Code window with a project named 'EncryptionProject'. The Explorer sidebar on the left lists files including 'encrypted_aes_key.bin', 'encrypted_fake_data.bin', 'encrypted_fake_message.bin', 'iv.bin', 'old', 'private_key.pem', 'public_key.pem', 'test_data', 'fake_data.json', 'fake_message.txt', 'decrypted_fake_data.json', 'decrypted_fake_message.txt', and 'main.py'. The main editor displays the 'main.py' file with the following Python code:

```
192 # معاً راجع نصيف كود عشاق نغزو نموي مطابقة للبيانات.
193
194 # اختيار النطاق بعد هذا التغيير
195 files = ['fake_data.json', 'fake_message.txt']
196 for file_name in files:
197     with open(f'test_data/{file_name}', 'rb') as original_file:
198         original_data = original_file.read()
199     with open(f'decrypted_{file_name}', 'rb') as decrypted_file:
200         decrypted_data = decrypted_file.read()
201
202     if original_data == decrypted_data:
203         print(f"✅ {file_name} matches the original exactly.")
204     else:
205         print(f"❌ {file_name} does NOT match the original.")
206
207
```

The bottom panel shows the 'TERMINAL' tab with the following output:

```
PS C:\Users\taeft\OneDrive\حبي\EncryptionProject> & C:/Users/taeft/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/taeft/OneDrive/حبي/main.py 1111.py"
enter the password:
```

We enter the required password, which is Strongpassword123.



This screenshot shows the same VS Code window, but the terminal now displays the password input:

```
enter the password:Strongpassword123
```

```
192 # واجهتيف كود عشاق لفقو نموي مطابقة لثبيانات  
193  
194 # اختيار النطاق بعد فك التشفير  
195 files = ['fake_data.json', 'fake_message.txt']  
196 for file_name in files:  
197     with open(f'test_data/{file_name}', 'rb') as original_file:  
198         original_data = original_file.read()  
199     with open(f'decrypted_{file_name}', 'rb') as decrypted_file:  
200         decrypted_data = decrypted_file.read()  
201  
202     if original_data == decrypted_data:  
203         print(f'✅ {file_name} matches the original exactly.')  
204     else:  
205         print(f'❌ {file_name} does NOT match the original.')  
206  
207
```

PS C:\Users\taeft\OneDrive\حفظي\EncryptionProject> & C:/Users/taeft/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/taeft/OneDrive/حفظي/EncryptionProject/main.py 1111.py"
enter the password:Strongpassword123
Key rotation done, old keys moved to keys/old/.
أج اجنب عيت الفول ا ففج و دي لوت مت
أج اجنب IV و AES ع اتقم دي لوت مت
أج اجنب ت افلمول ا ريفقت مت
أج اجنب IV و AES ع اتقم ففج و ريفقت مت
أج اجنب IV و AES ع اتقم ريفقت لاف مت
أج اجنب ت افلمول ا ريفقت لاف مت
✅ fake_data.json matches the original exactly.
✅ fake_message.txt matches the original exactly.
PS C:\Users\taeft\OneDrive\حفظي\EncryptionProject>

The outputs from the code :

- Old keys were moved to keys/old/ (key rotation completed).
- New RSA keys were generated and saved successfully.
- AES key and IV were generated successfully.
- Files were encrypted successfully.
- AES key and IV were encrypted and saved successfully.
- AES key and IV were decrypted successfully.
- Files were decrypted successfully.
- fake_data.json matches the original exactly.
- fake_message.txt matches the original exactly.

:Conclusion

In this project, we successfully implemented data encryption and decryption using modern encryption techniques such as RSA and AES, along with key rotation and .access control to ensure effective data protection

Sources

OWASP Cryptographic Storage Cheat Sheet - Verification of Decrypted Data-

OWASP Cryptographic Storage Cheat Sheet - Protecting Private Keys

OWASP Key Management Cheat Sheet

Cryptography Official Documentation - Private Key Encryption

OWASP Cryptographic Storage Cheat Sheet

IBM - Data Integrity Verification

[AWS - Key Rotation](#)